MULTI-AGENT AWS ARCHITECTURE SLIDE GENERATOR

Low-Level Technical Design and API Specification

Primary design constraint: Four specialized agents, coordinated by one central Step Functions state machine, producing editable PPTX slides.

Prepared for coding-agent driven implementation
Version 2.0 | 24 September 2026

1. Document Purpose

This document is an implementation-ready specification for a multi-user full-stack application that accepts RFP files and generates editable Business, Application and AWS Deployment Architecture slides. It translates the attached blueprint into a focused four-agent runtime model and defines service boundaries, APIs, persistence, orchestration, contracts, security and acceptance criteria.

2. Scope and Design Principles

- Exactly four logical AI agents in the slide-generation pipeline.
- AWS Step Functions is the central system orchestrator.
- Project, upload and knowledge-base functions are platform services, not extra agents.
- Tenant isolation is enforced at API, storage, retrieval and execution layers.
- All agent outputs are validated JSON contracts before downstream use.
- The final PPTX contains native PowerPoint shapes, connectors, text and image assets.

Blueprint alignment: Solution Architect -> Layout & Geometric -> Compiler -> Visual Reviewer. This sequence is preserved as the core generation pipeline.

3. Target Outcomes

Outcome	Definition of done
Multi-user project workspace	Authenticated users create isolated projects and upload multiple RFPs.
Grounded architecture generation	Solution topology is derived from project-scoped RFP retrieval plus approved shared references.
Three architecture types	Business, Application and Deployment slide requests use the same four-agent pipeline with different generation profiles.
Editable output	PPTX objects remain editable instead of being flattened into a single image.
Recoverable workflow	Each state is persisted, retryable and auditable through Step Functions and job records.

4. Logical Architecture

```
Web UI (React)
  | HTTPS / JWT
API Gateway + Cognito Authorizer
  |
Application API (FastAPI)
  |-- Project and document APIs
  |-- Job and artifact APIs
  |-- Starts Step Functions execution
  |
AWS Step Functions - CENTRAL ORCHESTRATOR
  |--> Agent 1: Solution Architect (LLM)
  |--> Agent 2: Layout & Geometric (LLM + deterministic validator)
  |--> Agent 3: Compiler (Python/python-pptx)
  |--> Agent 4: Visual Reviewer (VLM) -- correction loop --> Agents 2/3
  |
DynamoDB | S3 | Bedrock Knowledge Base | S3 Vectors | CloudWatch
```

4.1 Compute Limit: Maximum Four Runtime Services

Service	Deployment unit	Main responsibility	Agent hosted
API and Project Service	ECS Fargate service or Lambda/API Gateway	Authentication context, projects, uploads, KB sync status, jobs, artifacts	None
Architecture Intelligence Service	ECS Fargate service	RAG retrieval, prompt assembly, architecture model generation	Agent 1
Layout Service	ECS Fargate service	Coordinate generation, deterministic geometry validation and repair	Agent 2
Rendering and Review Service	ECS Fargate service with worker queue	PPTX compile, preview rendering, VLM review, export	Agents 3 and 4

Important: Step Functions is orchestration, not a fifth compute service. Managed stores and Bedrock services are also not application compute services.

4.2 Central Orchestrator Responsibilities

- Generate and propagate correlationId, userId, projectId and jobId.
- Wait for KB ingestion completion before architecture generation.
- Invoke four agents in order and validate each response.
- Retry transient failures with exponential backoff.
- Route validation failures to a controlled correction loop.
- Persist state transitions and terminal failure reasons.
- Prevent cross-project execution by checking immutable execution context.

5. Four-Agent Specification

#	Agent	Purpose	Input	Output	Runtime
1	Solution Architect	Create architecture topology grounded in project RFPs and approved references.	Generation request + retrieved evidence	architecture-model.json	Architecture Intelligence Service
2	Layout & Geometric	Convert topology into strict slide geometry, groups, lanes, connectors and z-order.	architecture-model.json + slide profile	layout-model.json	Layout Service
3	Compiler	Build editable PPTX using native shapes, connectors, text and AWS icons.	layout-model.json + icon manifest + theme	draft.pptx + preview images	Rendering and Review Service
4	Visual Reviewer	Assess preview against quality rules and produce machine-readable corrections.	Preview images + layout model + rules	review-report.json	Rendering and Review Service

5.1 Agent 1 - Solution Architect

Selects logical components and relationships. For deployment slides it includes AWS topology such as accounts, Regions, VPCs, Availability Zones, subnets, edge, compute, integration, data, observability and security controls. For Business and Application slides it uses corresponding semantic component types.

Validation gates:

- Every node has a stable ID.
- Every edge references valid source and target IDs.
- Each decision includes evidence IDs or is marked as an assumption.
- No coordinates or PPT-specific details are produced.

5.2 Agent 2 - Layout & Geometric

Maps the architecture graph to a 16:9 slide canvas. It calculates X/Y coordinates, dimensions, nested containers, alignment, spacing, connector routes and z-order.

Validation gates:

- All objects remain inside slide bounds.
- Children remain inside parent containers.
- No object overlaps unless overlap is explicitly permitted.
- Connector endpoints reference existing anchors.

5.3 Agent 3 - Compiler

Uses python-pptx to create native PowerPoint objects. It resolves the icon manifest, adds containers and labels, draws connectors, applies themes and writes speaker-note/source metadata where supported.

Validation gates:

- PPTX opens successfully.
- Mandatory slide title and legend exist.
- Shape counts match the layout model.
- All external assets resolve before compilation.

5.4 Agent 4 - Visual Reviewer

Uses rendered slide previews to identify clipping, low contrast, collisions, excessive crossings, inconsistent spacing and unreadable labels. It returns corrections rather than directly modifying the file.

Validation gates:

- Report conforms to schema.

- Each issue identifies object IDs.
- Severity and recommended action are present.
- Maximum review loops are enforced by the orchestrator.

6. End-to-End State Machine

```
VALIDATE_CONTEXT
  -> WAIT_FOR_KB_READY
  -> RETRIEVE_PROJECT_CONTEXT
  -> RUN_SOLUTION_ARCHITECT
  -> VALIDATE_ARCHITECTURE_MODEL
  -> RUN_LAYOUT_AGENT
  -> VALIDATE_LAYOUT_MODEL
  -> RUN_COMPILER
  -> RENDER_PREVIEW
  -> RUN_VISUAL_REVIEWER
  -> [PASS] PUBLISH_ARTIFACT
  -> [REVISE and attempt < max] APPLY_CORRECTIONS -> RUN_COMPILER
  -> [FAIL or max attempts] MARK_JOB_FAILED
```

State	Timeout	Retry	Persisted output
VALIDATE_CONTEXT	10s	No	validated execution context
WAIT_FOR_KB_READY	15m poll	Yes	ingestion status
RETRIEVE_PROJECT_CONTEXT	60s	2 retries	evidence bundle
RUN_SOLUTION_ARCHITECT	180s	2 retries	architecture model
RUN_LAYOUT_AGENT	120s	2 retries	layout model
RUN_COMPILER	300s	1 retry	draft PPTX and manifest
RUN_VISUAL_REVIEWER	180s	1 retry	review report
PUBLISH_ARTIFACT	60s	2 retries	versioned S3 artifact

Correction policy: Default maximum is two visual-review correction passes. Exceeding the limit fails the job with all intermediate artifacts retained for diagnosis.

7. REST API Contract

Base path: /api/v1. All endpoints require an access token. The backend derives userId/tenantId from verified claims; clients must not be trusted to supply identity scope.

Method	Endpoint	Purpose	Success
POST	/projects	Create a project	201
GET	/projects	List caller projects	200
GET	/projects/{projectId}	Get project and readiness state	200
PATCH	/projects/{projectId}	Update name or configuration	200
DELETE	/projects/{projectId}	Soft-delete project and schedule cleanup	202
POST	/projects/{projectId}/documents/presign	Create scoped upload URL	200
POST	/projects/{projectId}/documents/complete	Register uploaded object and start ingestion	202
GET	/projects/{projectId}/documents	List project documents	200
DELETE	/projects/{projectId}/documents/{documentId}	Remove document and update corpus	202
GET	/projects/{projectId}/knowledge-status	Get ingestion readiness	200
POST	/projects/{projectId}/slide-jobs	Start slide generation	202
GET	/projects/{projectId}/slide-jobs	List generation jobs	200
GET	/projects/{projectId}/slide-jobs/{jobId}	Get state and progress	200
POST	/projects/{projectId}/slide-jobs/{jobId}/cancel	Stop execution	202
POST	/projects/{projectId}/slide-jobs/{jobId}/retry	Create retry execution	202
GET	/projects/{projectId}/slide-jobs/{jobId}/artifacts	List outputs	200
GET	/artifacts/{artifactId}/download	Return short-lived download URL	200

7.1 Create Project

```
POST /api/v1/projects
Content-Type: application/json

{
  "name": "GIA Modernization",
  "description": "RFP-based architecture generation",
  "defaultThemeId": "aws-standard-v1"
}

201 Created
{
  "projectId": "prj_01J...",
  "status": "CREATED",
  "createdAt": "<ISO-8601>"
}
```

7.2 Request Upload URL and Complete Upload

```
POST /api/v1/projects/{projectId}/documents/presign
{ "fileName": "rfp.pdf", "contentType": "application/pdf", "sizeBytes": 2401234, "sha256": "..." }

200 OK
{ "documentId": "doc_01J...", "uploadUrl": "<presigned-url>", "objectKey":
"tenants/{tenantId}/projects/{projectId}/rfps/{documentId}/source/rfp.pdf", "expiresInSeconds": 900 }

POST /api/v1/projects/{projectId}/documents/complete
{ "documentId": "doc_01J...", "objectKey": "...", "etag": "..." }

202 Accepted
{ "documentId": "doc_01J...", "ingestionStatus": "QUEUED" }
```

7.3 Start Slide Job

```
POST /api/v1/projects/{projectId}/slide-jobs
Idempotency-Key: <uuid>

{
  "architectureType": "DEPLOYMENT",
  "instruction": "Create a highly available AWS deployment architecture",
  "documentIds": ["doc_01J..."],
  "slideProfile": "AWS_16_9",
  "outputFormat": "PPTX"
}

202 Accepted
{ "jobId": "job_01J...", "executionArn": "...", "status": "QUEUED", "statusUrl":
"/api/v1/projects/.../slide-jobs/job_01J..." }
```

7.4 Job Status and Error Model

```
GET /api/v1/projects/{projectId}/slide-jobs/{jobId}

200 OK
{
  "jobId": "job_01J...",
  "status": "VISUAL_REVIEW",
  "progressPercent": 85,
  "currentState": "RUN_VISUAL_REVIEWER",
  "attempt": 1,
  "createdAt": "<ISO-8601>",
  "updatedAt": "<ISO-8601>"
}

Error envelope
{
  "error": {
    "code": "LAYOUT_VALIDATION_FAILED",
    "message": "Two elements overlap",
    "correlationId": "corr_01J...",
    "details": [{"objectId":"node-api","rule":"NO_OVERLAP"}]
  }
}
```

HTTP	Usage
400	Invalid request/schema
401	Missing or invalid authentication
403	Project not owned by caller
404	Resource not found within caller scope
409	Invalid state or duplicate operation
413	File too large
415	Unsupported file type
422	Valid request but generation constraints cannot be satisfied
429	Tenant throttled
500	Unhandled internal error
502	Model/service dependency failed
504	Synchronous gateway timeout; job remains queryable

8. Canonical JSON Contracts

8.1 Architecture Model


```
{
  "schemaVersion": "1.0",
  "projectId": "prj_...",
  "jobId": "job_...",
  "architectureType": "DEPLOYMENT",
  "title": "Target AWS Deployment Architecture",
  "assumptions": [{"id":"asm-1","text":"...","approved":false}],
  "nodes": [
    {"id":"vpc-1","type":"CONTAINER","label":"Application VPC","parentId":null,"properties":
{"awsService":null},"evidenceIds":["ev-12"]},
    {"id":"ecs-1","type":"COMPUTE","label":"Application Service","parentId":"subnet-private-a","properties":
{"awsService":"Amazon ECS"},"evidenceIds":["ev-21"]}
  ],
  "edges": [{"id":"e-1","source":"alb-1","target":"ecs-1","protocol":"HTTPS","label":"routes traffic"}],
  "qualityAttributes":["availability","security","observability"]
}
```


8.2 Layout Model


```
{
  "schemaVersion":"1.0",
  "slide":{"widthIn":13.333,"heightIn":7.5,"safeMarginIn":0.25},
  "objects":[
    {"objectId":"ecs-1","kind":"ICON_LABEL","x":7.1,"y":3.0,"w":1.2,"h":0.9,"z":20,"parentObjectId":"subnet-
private-a","iconKey":"compute/ecs"}
  ],
  "connectors":
[{"edgeId":"e-1","sourceObjectId":"alb-1","targetObjectId":"ecs-1","route":"ORTHOGONAL","z":10}],
  "validation":{"overlapCount":0,"outOfBoundsCount":0}
}
```


8.3 Visual Review Report


```
{
  "decision":"REVISE",
  "score":82,
  "issues":[
    {"issueId":"vr-1","severity":"HIGH","rule":"TEXT_CLIPPED","objectIds":["ecs-1"],"recommendedAction":
{"type":"RESIZE","delta":{"w":0.25}}}
  ],
  "summary":"One label requires additional width"
}
```

9. Persistence Design

Table	PK/SK	Key attributes	Purpose
ArchitectureGenerator	PK=TENANT#{tenantId}; SK=PROJECT#{projectId}	ownerUserId, name, status, kbStatus, createdAt	Project record
ArchitectureGenerator	PK=PROJECT#{projectId}; SK=DOCUMENT#{documentId}	s3Key, sha256, ingestionStatus, sourceVersion	Document record
ArchitectureGenerator	PK=PROJECT#{projectId}; SK=JOB#{jobId}	executionArn, type, state, attempt, artifactIds	Job record
ArchitectureGenerator	PK=PROJECT#{projectId}; SK=ARTIFACT#{artifactId}	s3Key, mediaType, version, checksum	Artifact record
Idempotency	PK=TENANT#{tenantId}; SK=KEY#{idempotencyKey}	requestHash, responseRef, expiresAt	Duplicate request protection

9.1 S3 Object Layout

```
private-data-bucket/
  tenants/{tenantId}/projects/{projectId}/
    rfps/{documentId}/source/{fileName}
    rfps/{documentId}/metadata.json
    jobs/{jobId}/inputs/request.json
    jobs/{jobId}/models/architecture-model.json
    jobs/{jobId}/models/layout-model.json
    jobs/{jobId}/previews/attempt-{n}/slide-1.png
    jobs/{jobId}/reviews/attempt-{n}.json
    jobs/{jobId}/outputs/version-{n}/architecture.pptx

shared-reference-bucket/
  aws-icons/{version}/...
  architecture-patterns/{patternId}/...
  themes/{themeId}/theme.json
```

Icons and samples: Keep approved AWS icons and reusable reference architectures in a shared, read-only S3 reference bucket. Do not mix them with tenant RFP content or ingest raw icon binaries into the project KB. Store searchable text metadata for patterns separately.

9.2 Knowledge Base Isolation

- Every ingested chunk must carry tenantId, projectId, documentId and visibility metadata.
- Retrieval must apply tenantId AND projectId filters server-side.
- Shared reference content uses visibility=SHARED_APPROVED and a separate source prefix.
- The user instruction cannot override retrieval filters.
- Before prompt assembly, reject any evidence record whose metadata does not match execution context.

10. Internal Service Interfaces

These endpoints are private, authenticated with service identity and reachable only inside the application network. Step Functions normally invokes these services through controlled integrations or signed worker calls.

Service	Endpoint	Request	Response
Architecture Intelligence	POST /internal/v1/architectures/generate	execution context, evidence bundle, architecture type, instruction	architecture-model.json
Layout	POST /internal/v1/layouts/generate	architecture model, slide profile, theme	layout-model.json
Layout	POST /internal/v1/layouts/validate	layout model	validation report
Render/Review	POST /internal/v1/presentations/compile	layout model, icon manifest, theme	draft artifact manifest
Render/Review	POST /internal/v1/presentations/render-preview	draft artifact ID	preview manifest
Render/Review	POST /internal/v1/presentations/review	preview manifest, layout model	review-report.json
Render/Review	POST /internal/v1/presentations/apply-corrections	layout model, review report	revised layout model

10.1 Common Internal Headers

Header	Requirement
X-Correlation-Id	Required across every call
X-Tenant-Id	Injected by trusted orchestrator, never client-controlled
X-Project-Id	Must match payload and resource path
X-Job-Id	Required for idempotency and logging
Authorization	Short-lived service credential or SigV4 identity
Content-Type	application/json

11. Agent Prompt and Tool Boundaries

Agent	Allowed tools/data	Must not do
Solution Architect	Project-filtered retrieval, approved reference-pattern metadata, architecture schema validator	Generate coordinates, write PPTX, access other projects
Layout & Geometric	Architecture model, slide profile, icon dimensions, geometry validator	Change architectural meaning or add unapproved services
Compiler	Layout model, approved icon bucket, theme assets, python-pptx	Call LLM for architecture changes
Visual Reviewer	Rendered PNGs, layout model, design rules	Directly modify PPTX or change architecture semantics

11.1 Compiler Build Order

1. Create presentation and set 16:9 dimensions.
2. Create slide using selected master/layout.
3. Draw outer architecture zones and nested containers in z-order.
4. Add service icons and labels using stable object IDs.
5. Add connectors using precomputed routes and anchors.
6. Apply title, legend, theme fonts and color tokens.
7. Write artifact metadata and save to a temporary path.
8. Validate the package, upload draft, render preview and return manifest.

11.2 Deterministic Geometry Checks

- Bounds: x >= safeMargin, y >= safeMargin, x+w <= slideWidth-safeMargin.
- Containment: each child rectangle must fit inside its parent padding box.
- Collision: intersecting rectangles are rejected except for declared overlays.
- Minimum readable label size is controlled by slide profile.
- Connector crossing count is minimized; connectors must not pass through labeled objects.
- Stable IDs are preserved across correction cycles so issues remain traceable.

12. Security, Isolation and Governance

Control area	Implementation requirement
Authentication	Cognito or enterprise federation; API Gateway validates access tokens.
Authorization	Every resource lookup includes tenant and project ownership checks.
S3	Block public access; SSE-KMS; prefix-scoped roles; short-lived presigned URLs.
DynamoDB	Encryption enabled; conditional writes; least-privilege access to tenant-partitioned items.
Knowledge retrieval	Mandatory metadata filters and post-retrieval scope validation.
Prompt injection	Treat RFP content as untrusted data; system instructions and tool permissions remain immutable.
Network	Private subnets/endpoints where applicable; restrict service-to-service calls.
Secrets	Use Secrets Manager or workload identity; no static secrets in environment files.
Audit	Log actor, project, job, state, model request ID and artifact checksum without logging RFP contents.
Retention	Configurable retention; soft delete followed by controlled object and vector cleanup.

13. Reliability and Observability

- Structured JSON logs with correlationId, tenantId, projectId, jobId, agent and state.
- CloudWatch metrics: jobs started/completed/failed, state latency, model failures, review attempts and compile failures.
- Dead-letter handling for asynchronous events and exhausted retries.
- Idempotent job creation and artifact publishing.
- DynamoDB conditional updates prevent stale states from overwriting newer states.
- Alarms on failure rate, queue age, Step Functions failures and repeated cross-scope validation rejection.

13.1 Job Status Model

```
CREATED -> UPLOADING -> INGESTING -> READY -> QUEUED -> ARCHITECTING -> LAYOUT -> COMPILING -> VISUAL_REVIEW
-> COMPLETED | FAILED | CANCELLED
```

13.2 Failure Codes

Code	Meaning	Recovery
KB_NOT_READY	Selected documents are not searchable	Wait/poll, then resume
NO_RELEVANT_EVIDENCE	Retrieval returned insufficient scoped evidence	Return actionable validation failure
ARCHITECTURE_SCHEMA_INVALID	Agent 1 output violates JSON schema	Retry with schema errors
LAYOUT_VALIDATION_FAILED	Geometry checks failed	Repair through Agent 2
ICON_NOT_FOUND	Mapped service has no approved icon	Use generic service shape or fail by policy
PPTX_COMPILE_FAILED	Compiler/package failure	Retry once from persisted layout
VISUAL_REVIEW_REJECTED	Quality threshold not met	Correction loop up to configured limit
TENANT_SCOPE_MISMATCH	Retrieved or requested data violates context	Fail immediately and audit

14. Suggested Repository Structure

```
architecture-slide-generator/
  apps/web/
  services/api-project/
  services/architecture-intelligence/
  services/layout/
  services/render-review/
  packages/contracts/          # JSON schemas and generated models
  packages/pptx-engine/        # python-pptx primitives
  packages/observability/
  infrastructure/cdk/
  workflows/step-functions/
  tests/unit/
  tests/contract/
  tests/integration/
  tests/golden-slides/
  docs/adr/
```

15. Implementation Epics and User Stories

Epic	User story	Acceptance criteria
Project workspace	As a user, I can create multiple isolated projects.	Only owned projects are visible; IDs are immutable; operations are audited.
RFP ingestion	As a user, I can upload RFPs and see ingestion readiness.	Upload is direct-to-S3; checksum verified; status becomes READY only after successful sync.
Slide request	As a user, I can request Business, Application or Deployment architecture.	202 response includes job ID; duplicate idempotency key does not create another job.
Architecture model	As the system, I generate a valid, evidence-linked architecture graph.	Schema passes; node IDs unique; unsupported assumptions are flagged.
Layout	As the system, I create a collision-free 16:9 layout.	Bounds, containment, overlap and connector checks pass.
Editable PPTX	As a user, I download a PPTX with editable elements.	File opens; shapes/text/connectors remain editable; artifact checksum recorded.
Visual quality	As the system, I review and correct presentation defects.	Reviewer returns structured issues; workflow stops after configured attempts.

16. Test Strategy

Test layer	Coverage
Unit	Schema validation, tenant guards, geometry functions, icon mapping, PPT primitives.
Contract	Every internal endpoint validated against versioned request/response schemas.
Integration	S3 upload, DynamoDB persistence, KB readiness, Step Functions transitions, artifact retrieval.
Security	IDOR attempts, cross-project retrieval, malicious metadata, prompt injection in RFPs, presigned URL expiry.
Golden slide	Compare generated slide object graph and rendered preview against approved fixtures.
Failure recovery	Resume from persisted states after model, compiler and network failures.
Load	Concurrent projects, throttling behavior, queue backpressure and artifact generation.

17. MVP Definition

- One slide per job for Business, Application or Deployment architecture.
- PDF and DOCX RFP input; configurable file-size limit.
- One approved AWS theme and one 16:9 slide profile.
- Four-agent flow with a maximum of two review corrections.
- English labels and editable PPTX output.
- Project-scoped KB retrieval and shared approved references.
- Job status polling; notifications can be added later.

18. Coding-Agent Build Order

9. Create versioned JSON schemas first.
10. Implement project, document and job data access with tenant guards.
11. Implement presigned upload and ingestion status flow.
12. Create Step Functions state machine using stub workers.
13. Implement Agent 1 and architecture validation.
14. Implement deterministic layout engine and Agent 2 corrections.
15. Implement python-pptx compiler and icon resolver.
16. Implement preview rendering and Agent 4 report schema.
17. Connect correction loop, artifact publishing and download API.
18. Add contract, security, golden-slide and failure-recovery tests.

19. Explicit Architecture Decisions

Decision	Chosen approach	Reason
Central orchestrator	AWS Step Functions	Durable state, retries, branching and transparent execution history.
Number of logical agents	Four	Direct alignment with supplied blueprint and focused responsibilities.
Number of compute services	Maximum four	Avoid service-per-agent sprawl while retaining separate scaling characteristics.
Tenant KB strategy	Shared KB/vector infrastructure with mandatory project metadata filtering	Operationally simpler than per-user infrastructure; requires strict application-enforced isolation.
AWS icons	Shared read-only S3 asset repository	Reusable, versioned and independent from tenant data.
Reference architectures	Shared approved pattern repository plus searchable text metadata	Improves consistency without contaminating tenant corpus.
PPT generation	python-pptx native objects	Produces editable shapes, connectors, text and images.
Reviewer behavior	Returns corrections only	Preserves deterministic ownership of layout and compilation.

20. Definition of Done

- All public and internal APIs conform to documented schemas.
- Cross-project access tests fail closed.
- Step Functions can resume/retry without duplicating jobs or artifacts.
- All four agent outputs are versioned and validated.
- Generated PPTX opens in PowerPoint and remains editable.
- Visual reviewer can identify objects by stable IDs and issue corrections.
- Operational dashboards and alarms cover each workflow stage.
- README, local development configuration, deployment IaC and sample project fixtures are included.

Implementation note: Use this document as the coding agent source of truth. Any change to an API or JSON contract should update the schema version and associated contract tests.